

⚠️ Important

- Download and install Kiro IDE version 0.7 or later from [kiro.dev/downloads](#) 🔗

Kiro Powers - Empower Agents with Specialized Expertise

What Are Kiro Powers?

Kiro Powers are specialized capabilities that add expert context and tools to Kiro agents on-demand. Instead of loading all MCP (Model Context Protocol) tools into context immediately—which can overwhelm agents—Powers bundle MCP servers, steering guidance, and validation hooks into focused, reusable packages that activate only when relevant to your conversation.

The Problem with Traditional MCP

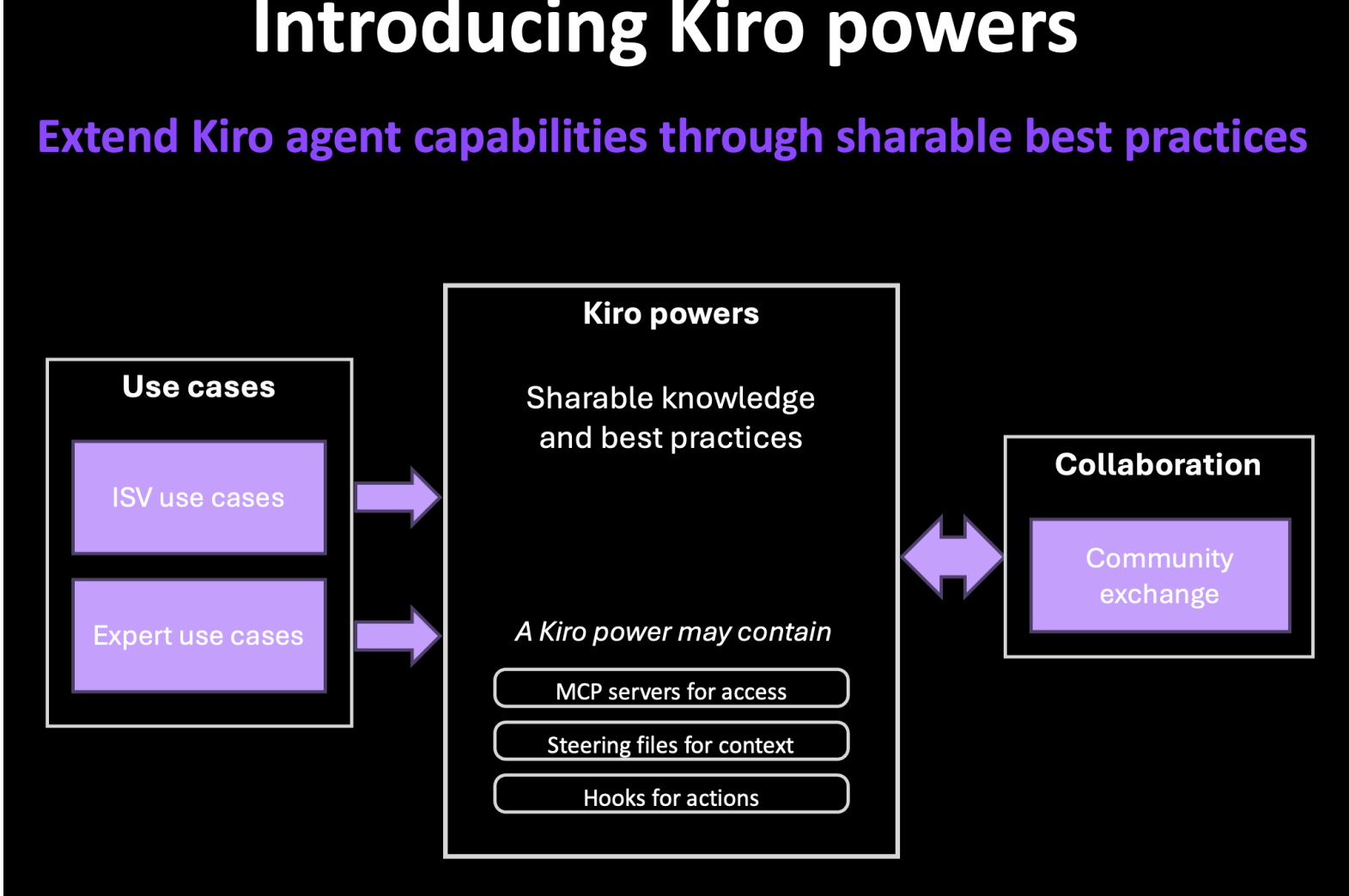
Traditional MCP servers load all their tools into the agent's context immediately, causing:

- **Context overload** - Too many tools confuse the agent
- **Reduced performance** - Large context windows slow down responses
- **Lack of focus** - Generic agents require extensive prompt engineering

How Powers Solve This

Powers provide:

- **On-demand activation** - Load only when conversation context requires them
- **Curated expertise** - Best practices from domain experts and tool providers
- **Single-click installation** - No manual MCP configuration needed
- **Stackable capabilities** - Use multiple Powers together for complex workflows



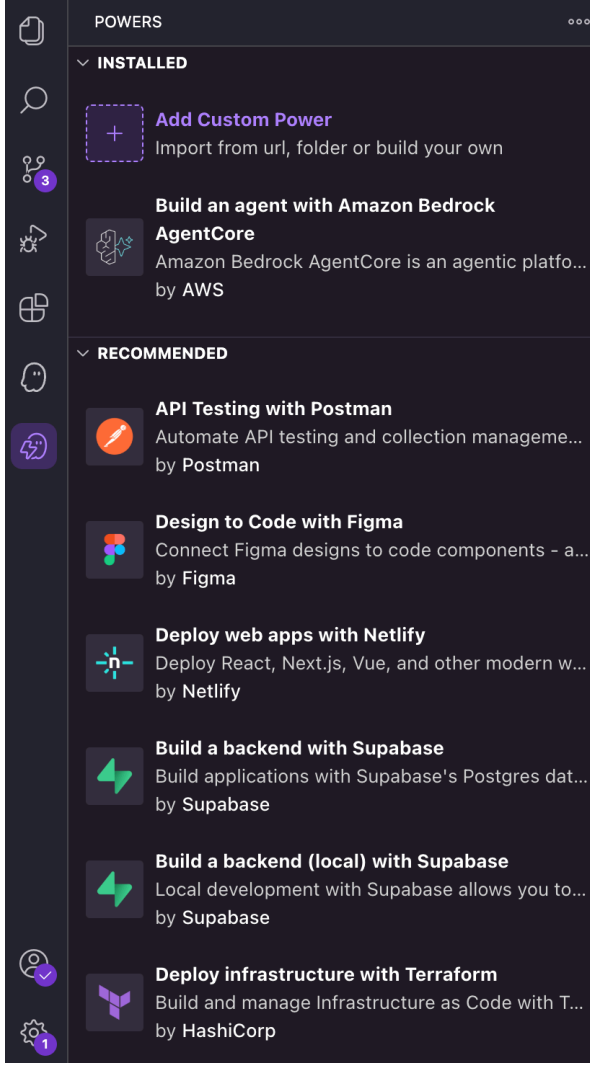
How Powers Work

1. Find or Create a Power

Browse curated Powers from AWS, partners, and the community at [kiro.dev/powers](#) 🔗, or build your own custom Powers for your organization's specific needs.

2. Install with One Click

Add Powers instantly from the website or IDE. Kiro handles all configuration and setup automatically, making specialized capabilities available in seconds.



3. Activate On-Demand

Powers load dynamically based on your conversation context. Kiro evaluates installed Powers and activates only the relevant ones, keeping your agent focused and efficient.

Available Power Categories

AI Agents

- **Build an agent with Amazon Bedrock AgentCore** - Build, deploy, and operate production-ready agents with memory, code execution, and observability
- **Build an agent with Strands** - Create AI agents using Bedrock, Anthropic, OpenAI, Gemini, or Llama models

Development & Deployment

- **Build full-stack apps with AWS Amplify** - Build and extend full-stack applications with Amplify Gen 2 using type-safe TypeScript
- **Build applications with Lambda durable functions** - Build resilient multi-step applications with automatic state persistence and retry logic
- **Build AWS infrastructure with CDK and CloudFormation** - Create well-architected AWS infrastructure with latest best practices, validate templates, and troubleshoot deployments
- **Deploy infrastructure with Terraform** - Manage Infrastructure as Code with Terraform registry and HCP Terraform
- **Deploy web apps with Netlify** - Deploy modern web apps to Netlify's global CDN
- **Deploy Web Apps and APIs to AWS using ECS Express Mode** - Deploy container images and get back an HTTPS endpoint
- **Build containers with Depot** - Write optimized Dockerfiles, configure CI/CD pipelines, and set up multi-platform container builds
- **InsideOut Cloud Infrastructure** - Design, price, and deploy production-ready AWS and GCP infrastructure through conversational AI
- **Ship software with Harness** - Interact with CI/CD pipelines, cloud cost management, security testing, chaos engineering, feature flags, and more
- **AWS Serverless Application Model (SAM)** - Opinionated guidance for development with AWS SAM

Databases & Backend

- **Build applications with Aurora PostgreSQL** - Leverage Aurora PostgreSQL-specific best practices
- **Build applications with Aurora DSQL** - Deploy serverless, PostgreSQL-compatible databases with scale-to-zero and pay-per-use pricing
- **Build a backend with Supabase** - Use Postgres database, authentication, storage, and real-time subscriptions (cloud and local development)
- **Build a database with Neon** - Serverless Postgres with database branching, autoscaling, and scale-to-zero
- **Build and Operate MSK Express Broker** - Create and operate Amazon MSK Express Brokers with best practices and monitoring

Design & Testing

- **Design to Code with Figma** - Connect Figma designs to code components automatically
- **Miro Board Context for Codegen** - Use Miro boards as the source of truth for architectural specs, UI diagrams, and project logic
- **API Testing with Postman** - Automate API testing and collection management
- **ScoutQA Testing** - AI-powered exploratory testing for web applications with automated bug detection and accessibility audits

Observability

- **AWS Observability** - Comprehensive platform combining CloudWatch Logs, Metrics, Alarms, Application Signals (APM), CloudTrail security auditing, and observability gap analysis
- **Datadog Observability** - Query logs, metrics, traces, RUM events, incidents, and monitors for production debugging
- **Dynatrace Observability** - Query logs, metrics, traces, problems, and Kubernetes events using DQL
- **Hud Runtime Code Sensor** - Detect errors, performance degradations, and CPU spikes in production with deep forensic context

Security

- **Snyk Secure at Inception** - Security scanning and remediation using Snyk, automatically scanning generated code for vulnerabilities
- **Checkmarx Application Security** - AI-powered security vulnerability detection and remediation for SAST, secrets, IaC, containers, and open source dependencies
- **AWS IAM Policy Autopilot** - Analyze application code to generate and deploy identity-based policies for application roles

Payments & SaaS

- **Stripe Payments** - Build payment integrations with subscriptions and billing
- **Checkout.com Global Payments** - Access comprehensive API documentation for payments, customers, disputes, and more
- **SaaS Builder** - Build production-ready multi-tenant SaaS applications with serverless architecture and integrated billing

Migration & Optimization

- **Plan and Migrate to Graviton** - Analyze source code for compatibility with Graviton processors (Arm64 architecture)
- **Perform Migration between Arm SoC** - Architecture-aware analysis and safe migration practices for embedded developers
- **GCP to AWS Migration Advisor** - Expert guidance for migrating workloads from Google Cloud Platform to AWS
- **Upgrade Spark applications on AWS** - Accelerate Apache Spark version upgrades with automated code transformation and validation
- **Troubleshoot Spark applications on AWS** - Analyze failures and identify root causes on EMR, Glue, and SageMaker
- **AWS Cost Optimization** - Analyze spending, get optimization recommendations, and build cost-aware applications

Specialized

- **Build geospatial applications with Amazon Location Service** - Add maps, places search, geocoding, routing, and other geospatial features
- **AWS HealthOmics** - Create, migrate, run, debug, and optimize genomics workflows
- **OpenSearch** - Accelerate proof-of-concept search applications with guided, end-to-end architecture planning

Stacking Powers

Use multiple Powers together to create enhanced capabilities. For example:

- **Stripe + Dynatrace** - Build payment systems while monitoring real-time performance
- **AgentCore + Aurora PostgreSQL** - Create agents with persistent database backends
- **Figma + Netlify** - Design-to-deployment workflow for web applications
- **Snyk + Harness** - Secure code scanning integrated into your CI/CD pipeline
- **Amplify + Supabase** - Full-stack frontend with a powerful Postgres backend

Creating Your Own Powers

Build custom Powers for your organization by:

1. Configuring MCP connections
2. Adding steering guidance
3. Defining validation hooks
4. Sharing via GitHub with "Add to Kiro" button

You can also install the **Build a Power** Power from [kiro.dev/powers](#) 🔗 for templates, best practices, and validation when creating your own Powers.

Learn more at [kiro.dev/docs/powers/create/](#) 🔗

In This Lab

Section	What You'll Do	Time
Build an Agent with Amazon Bedrock AgentCore	Install the AgentCore Power, create and deploy an AI agent, build a chat web UI	~45 min
Build & Use a UI Tester Power	Use the "Build a Power" power to create a custom UI Tester power, then test a web app with it	~30 min

Next Steps

Continue to the first hands-on lab to build your first agent with the AgentCore Power.